As a DevOps Architect, I have designed a robust, secure, and compliant infrastructure for a **Banking Management System**. Given the sensitive nature of banking data, this architecture prioritizes high availability, data integrity, and strict compliance (PCI-DSS/GDPR readiness).

---

1. Deployment Architecture

- **Cloud Provider:** AWS (Recommended for regulatory compliance).
- **Infrastructure as Code (IaC):** Terraform.
- **Pattern:** Multi-AZ deployment within a Virtual Private Cloud (VPC).
- **Network:**

* **Public Subnet:** Application Load Balancer (ALB).

* **Private Subnet:** Backend services (ECS/Fargate) and RDS instances.

* **Isolation:** WAF (Web Application Firewall) to protect against common web exploits (SQLi, XSS).

2. Docker

- **Base Image:** `alpine-openjdk-17` (or similar distroless/slim images) to minimize attack surface.
- **Multi-stage builds:**

Stage 1:* Maven/Gradle build.

Stage 2:* Runtime image.

- **Security:** Scan images for vulnerabilities using **Trivy** during CI/CD.

3. CI/CD (GitHub Actions)

- **Workflow:**

1. **Commit/Push:** Trigger pipeline.

2. **Test:** Execute unit/integration tests with JaCoCo coverage reports.

3. **Security:** SAST (SonarQube) and Container Scanning.

4. **Build:** Create Docker image, tag with git-sha, push to ECR (Elastic Container Registry).

5. **Deploy:** Update ECS Service using a **Blue/Green deployment** strategy to ensure zero downtime.

4. Hosting

- **Compute:** AWS ECS (Elastic Container Service) with **AWS Fargate** (Serverless, no managing EC2 instances).
- **Load Balancing:** AWS ALB (Application Load Balancer) handling TLS termination.

5. Database Hosting

- **Engine:** Amazon RDS (PostgreSQL with Multi-AZ enabled).
- **Encryption:** AES-256 at rest (AWS KMS) and SSL/TLS in transit.
- **Network:** No public IP; accessible only via Security Groups from ECS tasks.

6. Environment Variables

- **Management:** AWS Systems Manager (SSM) Parameter Store or AWS Secrets Manager.
- **Sensitive Data:** DB credentials, API keys, and JWT secrets injected at runtime via Secrets Manager.
- **Non-Sensitive:** `LOG_LEVEL`, `ENVIRONMENT` (Prod/Staging).

7. Monitoring

- **APM:** AWS X-Ray or Datadog for distributed tracing (essential for banking transaction flow).
- **Metrics:** Amazon CloudWatch (CPU, Memory, Request Count, 5xx Error rates).
- **Alerting:** PagerDuty integration for critical alerts (e.g., Database connection failures).

8. Logging

- **Centralized:** AWS CloudWatch Logs.
- **Retention:** 1–7 years (depending on local financial data retention laws).
- **Format:** JSON structured logging for easy ingestion into ELK Stack or Splunk.
- **Audit Trail:** AWS CloudTrail enabled for all API calls within the environment.

9. Scaling

- **Horizontal Pod Autoscaling (HPA):** ECS Service Auto Scaling based on CPU/Memory utilization (target 70%).
- **Database:** Read Replicas to handle heavy reporting traffic, keeping the Primary instance dedicated to Write/Transactional operations.

10. Backup Strategy

- **Database:**

* Daily automated snapshots.

* Point-in-Time Recovery (PITR) enabled for up to 35 days (allows restoration to a specific second).

- **Infrastructure:** Terraform state files stored in S3 with versioning and object locking enabled.
- **Disaster Recovery (DR):** Cross-region backup replication of RDS snapshots to a secondary AWS region.

Summary Checklist for Security

| Layer | Solution |

| :--- | :--- |

| **Encryption** | TLS 1.3 everywhere; KMS for all disks/databases. |

| **Identity** | IAM Roles for Tasks (Principle of Least Privilege). |

| **Access** | VPN access only for internal management (AWS Client VPN). |

| **Compliance** | Audit logs for every transaction performed via API. |